

Codeforces Mashup Generator

Brief: Concise Technical Overview

Prepared for Salman Adnan's portfolio

Scope

Concise, accurate overview of `codeforces-mashup-generator`, a single-file (229-line) Python CLI script by Salman Adnan. Source: `generate_mashup.py`, `README.md`, `requirements.txt`, `.env.example`. For a full, beginner-safe, function-by-function walkthrough with every term defined, see the companion `deep-dive.pdf`.

1 What It Does

The script builds fair **mashup contest** problem sets for a Codeforces study group. Given a set of rating (difficulty) bands, it queries the public Codeforces API and returns, per rating, a batch of candidate problems that:

- no listed group member (`--exclude-users`) has solved (an “Accepted” verdict), so no one gets a problem they already know;
- optionally, at least one trusted, strong reference solver (`--include-users`, 46 well-known competitive programmers by default) *has* solved, as a rough solvability/quality signal;
- optionally match a tag rule (`--include-tags/--exclude-tags/--tag-mode`).

Results are written to an Excel sheet (rating, contest id, index, name, tags, direct link). This is solo work by Salman Adnan; MIT licensed.

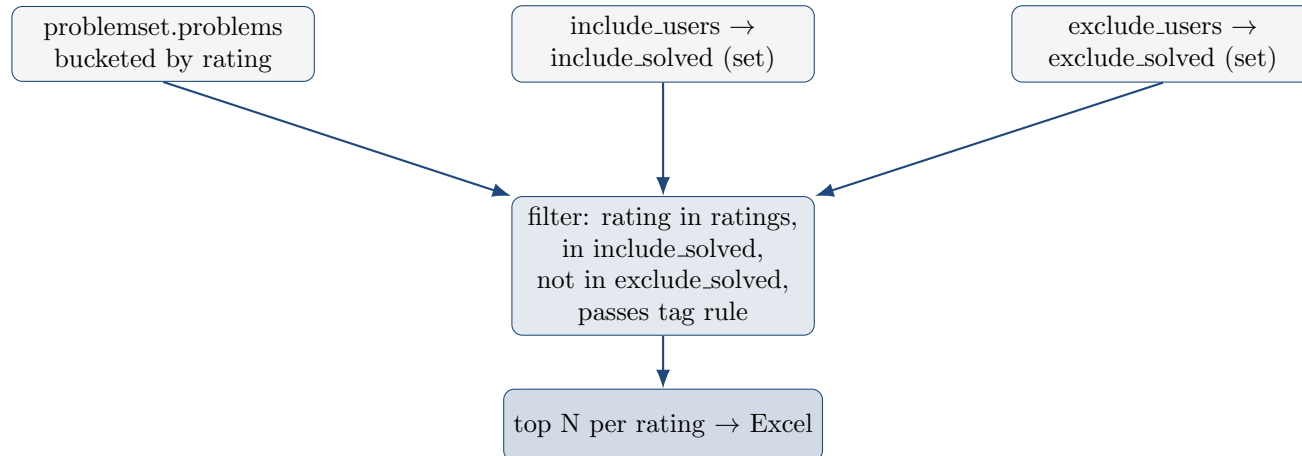
2 Architecture

One entry point, five sequential stages:



Two Codeforces endpoints supply all data: `problemset.problems` (the entire problem archive, once) and `user.status` (one user's full, paginated submission history, called once per handle in the include and exclude lists).

2.1 Data flow



Both solved sets are built by one shared helper, `collect_solved(handles, delay)`, called twice (once per handle list) — a refactor the author made specifically so the “`verdict == OK`” rule and the dedup logic exist in exactly one place, per the README.

3 Function Summary

Function	Role
<code>parse_args()</code>	Defines every CLI flag via <code>argparse</code> ; auto-generates <code>--help</code> with defaults shown.
<code>fetch_problemset(lang, delay)</code>	One call to <code>problemset.problems</code> ; returns the full problem archive as a JSON list. Checks both HTTP-level and API-payload-level status.
<code>fetch_user_status(handles, delay)</code>	Paginates through <code>user.status</code> (10,000 rows/page) until a short page signals the end; per-handle exceptions are caught and logged, not fatal.
<code>collect_solved(handles, delay)</code>	Reduces every OK-verdict submission across a handle list to a <code>set</code> of <code>(contestId, index)</code> tuples — the canonical identity used to compare problems across the two differently shaped API responses.
<code>get_filtered_problems(args)</code>	Orchestrates: bucket by rating, build both solved sets (unless disabled by <code>--no-include-filter/--no-exclude-filter</code>), keep only problems passing all active filters.
<code>passes_tag_filter(problem)</code>	Always drops excluded tags; if <code>--include-tags</code> and <code>--tag-mode != off</code> , applies <code>some</code> (intersect), <code>all</code> (subset), or <code>strict</code> (set equality).
<code>pick_choices_per_rating(filtered)</code>	Picks <code>n</code> per rating — first <code>n</code> survivors in API order (not randomised; see limitations).

Function	Role
<code>save_to_excel(choices, filename)</code>	Flattens selections into rows, builds a pandas <code>DataFrame</code> , writes via <code>openpyxl</code> . No-op with a printed message if nothing survived.

4 Key Design Decisions

- **Canonical identity across two API shapes.** `problemset.problems` returns flat problem objects; `user.status` wraps the problem under a nested `problem` key and sometimes omits its `contestId`, requiring a `prob.get('contestId')` or `s.get('contestId')` fallback. Normalising both to the `(contestId, index)` tuple is what makes set membership checks correct.
- **Two-layer API error handling.** An HTTP 200 response can still carry `{"status": "FAILED"}` in its JSON body (e.g. unknown handle); the code checks both the transport layer (`raise_for_status()`) and the payload's own `status` field.
- **Sets for $O(1)$ membership.** `include_solved` / `exclude_solved` are built once as Python sets so every per-problem check is a fast lookup rather than a rescan of submission lists.
- **Per-handle fault isolation.** Each network call is wrapped in a `try/except`; one bad or renamed handle does not abort the whole run — confirmed live when two stale default exclude-handles returned HTTP 400 and the run still produced a correct sheet.
- **Refactor from hardcoded config to CLI.** A leftover file, `all_users.txt`, preserves what looks like the pre-`argparse` hardcoded handle lists; it is not read by the script at runtime (see Flagged box).

Flagged / Inferred

`all_users.txt` is inferred, from its content and a `# === CONFIGURATION ===` header, to be a stray artifact of the pre-refactor hardcoded version described in the README — the README does not name this file directly, and it is functionally inert (never imported).

5 Verified Run

```
python generate_mashup.py --include-users ksun48 \
  --exclude-users salmanadnan2025 Musab1Blaser \
  --ratings 800 1200 1600 --choices-per-rating 3 \
  --include-tags greedy --tag-mode some -o mashup_test.xlsx
```

Per the README: completed in about 14 seconds against the live API, producing 9 rows (3 per rating), all tagged **greedy**, none solved by the two excluded members, all solved by **ksun48**; two stale default-list handles returned HTTP 400 without stopping the run.

6 Limitations (author’s own assessment, from the README)

- Downloading every submission of every include-handle (tens of thousands for prolific users) to answer a yes/no solved question is heavier than needed; caching or using the archive’s own `solvedCount` field would be far cheaper.
- Selection takes the newest `n` survivors in API order, not a random sample — repeat runs with the same filters return near-identical sheets.
- A failed `user.status` call for an *excluded* member silently weakens the fairness guarantee instead of hard-failing.
- Diagnostics go to `stdout` via `print` and the exit code is always 0, so the script cannot be safely chained in an automated pipeline yet.
- `--choices-per-rating` is a single global value; a per-rating override is not currently supported.

7 Dependencies

`requests` (HTTP client), `pandas` (tabular data + Excel export), `openpyxl` (Excel-writer engine used by `pandas`). No API key or environment variables required; all configuration is via CLI flags.